BENEFITSCOPE

The codebase, in plain language

A careful clerk that reads a credit card's own benefit guide and tells you, yes or no, whether a purchase is covered - and shows you the exact page, so it never has to be taken on faith.

What it covers: purchase protection, return protection, trip-delay insurance.
Cards: Chase Sapphire Preferred, American Express Platinum, Citi Strata Premier, Capital One Venture X.
Scope: a mid-term prototype. Purchases are sample or typed by the user; there is no live bank connection.

The big idea

Credit cards promise things like "if your new TV is stolen, we will pay you back." But the real rules live in long PDF booklets, and every card sets different numbers. Most people cannot tell whether a theft, a refused return, or a delayed flight is actually covered.

BenefitScope reads those booklets for you. It answers one question - *is this purchase covered, yes or no* - and it always shows the exact sentence from the booklet behind the answer. The guiding rule of the whole project: **never guess.** If a page does not state a number, the system refuses to invent one.

The two questions it answers

- **Before you buy:** "I want to buy a Sony TV from xyz store - what does my card cover?" It returns the matching guide pages.
- **After something happens:** theft, damage, a denied return, or a delayed flight - "can I make a claim?" It returns a yes or no with the reason and the source page.

How it is built: the pieces

Think of the project as rooms in a house. Each room does one job, and they hand work to each other in a line.

1. The booklets - the raw material

- `sources.json` lists 13 real card booklets to fetch.
- `download_guides.py` downloads those PDFs.
- `ingest.py` splits each PDF into small page-by-page pieces and remembers which card and page each piece came from. The result is saved to `data/chunks.json`.

2. The search - finding the right sentence

- `retriever.py` is a simple search engine over those pieces. Ask a question and it returns the most relevant paragraphs, each with a page number and a link. It uses an old, transparent method (TF-IDF) on purpose, so you can always see why a page was chosen.

3. The hard rules - facts that need no opinion

- `benefits_data.json` stores the exact numbers for each card: the time window, the dollar cap, the delay hours, and the documents required (11 rule records).
- `eligibility/rules.py` is the rulebook checker. It does the black-and-white checks: right card? inside the time window? police report provided for a theft? These are pure logic - no AI.
- `eligibility/prefilter.py` is a quick bouncer. If a purchase cannot possibly qualify (wrong card, too late, wrong category), it is rejected at once and the AI is never called. This saves money and prevents mistakes.

4. The judge - where AI comes in, carefully

- `eligibility/reasoner.py` makes the decision. It runs the bouncer, pulls the matching booklet sentences, then sends the hard numbers *and* those sentences to an AI model (Groq) with strict instructions: use only what you are given, never invent a number, and if documents are missing the answer is no.
- A safety net: if the AI ever disagrees with the rulebook in a suspect way - for example, demanding a document the event does not need - the code trusts the rulebook, not the AI.

5. The shopping helper

- `eligibility/purchase_lookup.py` reads a sentence like "buy a Sony TV from xyz store," works out that it is electronics, and finds the booklet pages about purchase protection and returns - quoting a discount only if a page actually states one.

6. Accounts, history, and samples

- `accounts/store.py` handles sign-up and login. Passwords are stored safely (a salted hash, never plain text). Each account sees only the cards that person picked.
- `data/generate_transactions.py` creates 400 made-up purchases, each already labelled yes or no from the rulebook, so accuracy can be measured. `eval/` runs those checks.

7. The two front doors

- `frontend/app.py` is the website (built with Streamlit) - the tabs for checking purchases, browsing rules, and searching guides.
- `api/main.py` is the machine-to-machine version (FastAPI). Another program can send a purchase and get the same yes/no decision back as data.

How one check flows, end to end

When you ask whether a purchase is covered, the work moves through five stages. The first three never call the AI.

1. **Quick bouncer (prefilter)**
   Checks the hard facts: right card, the event matches the benefit, the category is allowed, the date or delay is inside the rule, and the card was actually charged. If it cannot qualify, the answer is "No" with a plain reason - and we stop here.
2. **Find the page (retriever)**
   If it still looks possible, the search engine pulls the matching booklet page, plus the next page, because exclusions often sit there.
3. **Grounded decision (the model)**
   The hard numbers and the short quote go to the AI with strict rules. It replies with a yes/no, a confidence, the quoted sentence, and the source page link.
4. **Safety checks**
   The reply is compared against the rulebook. Missing required documents force a "No." A model demand for an unneeded document is ignored in favour of the rule.
5. **The verdict**
   You see the stamp: "May qualify" or "Does not qualify," with the reason, the quoted sentence, and a link to the exact page.

The one thing that makes it trustworthy

Everywhere, the design refuses to guess. The hard numbers come from a rulebook; every quote is tied to a real page you can open; and the AI is boxed in so it cannot invent coverage. It is also honest where a benefit is absent - for example, Citi Strata Premier has no return protection, and the system says so instead of filling the gap.

File map

Where each job lives in the project.

File	What it does
`sources.json`	The registry of 13 official guide PDFs to download.
`download_guides.py`	Downloads the registered PDFs.
`ingest.py`	Splits PDFs into page-level chunks (data/chunks.json).
`retriever.py`	TF-IDF search over the chunks; returns page + link.
`benefits_data.json`	The 11 exact numeric rules: window, cap, delay, documents.
`eligibility/rules.py`	Deterministic rulebook checks and the ground-truth labels.
`eligibility/prefilter.py`	Rejects impossible claims before any AI call.
`eligibility/reasoner.py`	Full decision: filter, retrieve, ask the model, safety-check.
`eligibility/purchase_lookup.py`	Turns a shopping sentence into guide quotes.
`llm_policy.py`	Talks to the Groq model and loads the API key.
`accounts/store.py`	Sign-up and login (salted hash); per-user purchases.
`data/generate_transactions.py`	Builds 400 labelled sample purchases.
`data/transactions.json`	The 400 sample purchases with answers.
`data/users.json`	Saved accounts (password hashes only).
`eval/ragas_eval.py`	Measures retrieval quality and eligibility accuracy.
`frontend/app.py`	The Streamlit website (the user interface).
`api/main.py`	FastAPI endpoints for other programs.

The tools used

- **Python** for everything.
- **pypdf** to read the PDFs (some Chase files also needed **cryptography**).
- A **TF-IDF** search written inside the project - no heavy embedding index.
- **Groq** chat model (openai/gpt-oss-20b), temperature 0, JSON output.
- **FastAPI** for the data endpoints, **Streamlit** for the website.
- **PBKDF2-SHA256** with a random salt for passwords; all data stays in local files.